This lecture introduces the concept of Object-Oriented Programming (OOP) in Python, emphasizing its importance in structuring code, enhancing reusability, and facilitating easier modifications and feature additions. The lecture covers key OOP concepts such as classes, objects, encapsulation, inheritance, and abstraction, with a primary focus on classes and objects.

Key topics covered:

- Importance of OOP: OOP provides structure, increases code reusability, and simplifies modifications.
- Classes:
    - A class is defined as a blueprint or a sketch for creating objects.
    - Python inherently understands built-in classes like `int` and `str` when variables are created (e.g., `i = 10` creates an integer object).
    - Custom classes can be created using the `class` keyword (e.g., `class Test:`).
    - The `pass` keyword is used when a class has no initial implementation.
- Objects:
    - Objects are instances or variables of a class.
    - Creating an object involves instantiating a class (e.g., `c = Course()` creates an object `c` of the `Course` class).
- Methods within a Class:
    - Functions defined inside a class are called methods.
    - The `self` parameter is crucial; it refers to the instance of the class and attaches the method to the class.
    - Without `self`, the method cannot be properly accessed via an object.
- Class Variables/Object Instances:
    - Multiple objects can be created from a single class (e.g., `c1 = Course()`, `c2 = Course()`).
    - Each object is a separate instance, allowing for reusability of the class methods.
- Passing Arguments to Methods:
    - Arguments can be passed to methods within a class, similar to regular functions (e.g., `def course_details(self, course_name, course_mentor):`).
    - The `self` parameter remains the first parameter, acting as a reference to the class instance.
- Passing Data During Object Creation (The `__init__` Method):
    - The `__init__` method is a special method used to pass data when creating an object.
    - It is automatically called when an object is instantiated.
    - Parameters passed during object creation are received by the `__init__` method (e.g., `class Course: def __init__(self, name, mentor):`).
    - Inside `__init__`, `self.name = name` and `self.mentor = mentor` attach the passed data to the object.
- Accessing Variables:

    - Outside the class, variables are accessed using the object name (e.g., `c.name`).
    - Inside the class methods, instance variables are accessed using `self.variable_name` (e.g., `self.name1`).
- Variable Naming:
    - The names of the parameters in `__init__` and the instance variables (e.g., `self.name1`) do not have to be the same, but it's crucial to understand how data is passed and attached to the object.
- Using Instance Variables within a Class:
    - Instance variables (created in `__init__`) are accessed within class methods using `self` (e.g., `self.name1`, `self.mentor1`).

Main Learning Objectives and Takeaways:

- Understand the fundamental concepts of classes and objects in Python.
- Learn how to create a class and instantiate objects from it.
- Grasp the significance of the `self` parameter in class methods.
- Know how to use the `__init__` method to pass data during object creation.
- Understand how to access instance variables both inside and outside the class.
- Appreciate the reusability aspect of OOP through classes and objects.

The lecture concludes by encouraging viewers to complete assignment questions to reinforce their understanding and gain practical experience with classes and objects in Python.

-